

Camera 约拍服务平台—SOLID 检查清单

一、实验流程

1. 向 AI 提供 P1 需求规格说明书和 P2 架构设计文档，由 AI 生成核心类图设计方案。
2. 团队对照 SOLID 五项原则检查类职责、扩展方式、继承关系、接口边界和依赖方向。
3. 将发现的问题落实到核心类图和设计文档中，保证类图、API、ER 图和建表 SQL 使用同一套业务口径。

二、SOLID 检查总表

SOLID 原则	检查问题	是否存在问题	问题说明	处理方案
S - 单一职责	有没有类承担过多职责？	是，1 处	Order 同时承担订单状态流转、支付结果处理、自动确认、申诉状态进入、结算和退款等职责，容易让订单实体包含过多流程编排逻辑。	保留订单状态转换领域方法；支付回调、自动确认、申诉记录创建、结算和退款执行由 Service 层触发。Order 只负责校验当前状态并转换自身状态。
O - 开闭原则	状态扩展是否需要大面积修改现有代码？	是，1 处	AI 生成设计只覆盖顺利履约链路，缺少取消、仲裁、返修、退款等异常分支。订单状态转换规则也容易集中堆叠在一处。	订单状态统一为 10 个状态，转换规则按当前状态分组，避免把全部流转写成一条线性流程。
L - 里氏替换	子类型是否能稳定替换父类型使用？	是，1 处	学生认证和服务方实名认证虽然都有申请、通过、拒绝动作，但材料数量、字段含义和校验规则不同。如果强行抽成完全统一的认证父类，容易造成替换语义不稳定。	两类认证保持独立实体，共用 CertStatus 状态枚举；后台审核在 Service 层按统一操作入口处理，不强制引入认证继承层级。
I - 接口隔离	类是否暴露了调用方不需要的方法？	是，1 处	User 如果包含注册、登录、验证码校验、Token 签发等认证流程，会让订单、消息、评价等模块看到不需要的认证能力。	User 只保留资料维护、角色切换、信用等级计算等用户实体职责；登录注册由 AuthService 和会话接口实现。
D - 依赖倒转	高层业务是否依赖低层存储实现？	是，1 处	作品图片、交付文件、认证材料如果直接保存公开 URL，会让业务层依赖具体文件存储方式，也不利于私有文件权限控制。	领域类统一通过 fileId 关联 StoredFile；文件上传、临时访问地址、权限校验由 FileStorageService 及基础设施层处理。

AI 生成设计中发现的 SOLID 问题总数：5 处。

三、逐项检查记录

3.1 S: Order 职责边界

问题描述

AI 生成设计中，Order 同时包含创建订单、支付、取消、开始拍摄、确认交付、自动完成、进入申诉、结算、退款等方法。订单确实是交易主实体，但如果把定时任务、支付回调、申诉创建和结算执行都放入订单实体，会导致单个类承担流程编排职责。

处理方案

Order 保留以下领域方法：

- create(quote)：根据确认后的报价生成订单。
- pay()：将订单从待支付推进到已支付待拍摄。
- cancel(reason)：处理允许取消场景下的订单取消。
- startShooting()：进入拍摄阶段。
- markPendingDelivery()：拍摄完成后进入待交付。
- confirmDelivery()：需求方确认收货并完成订单。
- autoComplete()：由定时任务触发，订单实体只执行状态校验与完成状态转换。
- applyDispute()：由申诉服务触发，订单实体只进入 APPEALING 状态。
- settleToProvider()：结算服务触发后更新结算相关状态。
- refund(amount, reason)：退款服务触发后更新退款相关状态。

Service 层负责调用外部支付、对象存储、定时任务、通知、申诉记录和结算逻辑，Order 不直接依赖这些基础设施。

落实结果

订单实体的职责限定为“维护自身交易状态和资金状态”，流程编排保留在 Service 层，符合单一职责原则。

3.2 O: 订单状态流转

问题描述

AI 生成设计倾向于把订单设计成顺利流程：待支付、已支付、拍摄中、已交付、已完成。该流程无法表达支付前取消、支付后取消、待交付、仲裁、返修、退款等约拍平台核心异常场景。

处理方案

订单状态统一为：

```
PENDING_PAYMENT,  
PAID_PENDING_SHOOT,  
SHOOTING,  
PENDING_DELIVERY,  
DELIVERED_PENDING_CONFIRM,  
APPEALING,  
REWORK_REQUIRED,  
COMPLETED,  
CANCELLED,  
REFUNDED
```

状态转换规则按当前状态分组：

```
public Set<OrderStatus> getAllowedTransitions() {  
    switch (this) {  
        case PENDING_PAYMENT:  
            return Set.of(PAID_PENDING_SHOOT, CANCELLED);  
        case PAID_PENDING_SHOOT:  
            return Set.of(SHOOTING, CANCELLED, APPEALING);  
        case SHOOTING:  
            return Set.of(PENDING_DELIVERY, APPEALING);  
    }  
}
```

```

    case PENDING_DELIVERY:
        return Set.of(DELIVERED_PENDING_CONFIRM, APPEALING);
    case DELIVERED_PENDING_CONFIRM:
        return Set.of(COMPLETED, APPEALING, REWORK_REQUIRED);
    case APPEALING:
        return Set.of(COMPLETED, REWORK_REQUIRED, REFUNDED);
    case REWORK_REQUIRED:
        return Set.of(PENDING_DELIVERY, APPEALING);
    default:
        return Set.of();
}
}

```

落实结果

订单状态与 API 规范、核心类图、ER 图和建表 SQL 保持一致，能够覆盖正常履约和异常纠纷分支。

3.3 L：认证模型替换关系

问题描述

学生认证和服务方实名认证都具有申请、通过、拒绝等动作，但二者材料结构不同：学生认证关注学校、学号和学生证材料；服务方实名认证关注真实姓名、身份证号和身份证正反面材料。如果强行把二者抽成完全一致的父类，后台处理时可能误以为两种认证材料可以无差别替换。

处理方案

- StudentCertification 和 RealNameCertification 保持独立实体。
- 二者共用 CertStatus：PENDING_REVIEW / APPROVED / REJECTED。
- 审核操作由后台审核 Service 统一组织，但每类认证保留自身字段和校验规则。

落实结果

认证模块避免了不稳定的继承关系，同时保留统一审核入口，符合里氏替换原则和最小复杂度原则。

3.4 I：User 接口边界

问题描述

AI 生成设计把注册、登录等认证流程放入 User，会导致用户实体接口过宽。订单、会话、评价等模块只需要用户 ID、角色、昵称、头像、信用分等基础信息，并不需要直接依赖登录注册行为。

处理方案

User 只保留用户实体职责：

- switchRole(role)
- updateProfile(dto)
- getCreditLevel()

短信验证码、登录注册、Token 签发、退出登录等流程通过认证服务和会话接口完成。

落实结果

用户实体的接口更小，业务模块只依赖自身需要的用户信息，符合接口隔离原则。

3.5 D：文件存储依赖方向

问题描述

约拍平台中存在头像、作品集、证件材料、需求参考图、订单交付作品、申诉证据等多类文件。如果领域对象直接保存文件访问 URL，会把业务设计绑定到具体存储实现，也会削弱私有文件访问控制能力。

处理方案

- 建立 StoredFile 作为统一文件元数据类。
- 作品图片、服务橱窗封面、认证材料、交付作品、申诉证据统一通过 fileId 关联 StoredFile。
- 文件本体保存在本地文件系统、MinIO 或对象存储中。
- 上传、权限校验、临时访问地址生成由 FileStorageService 和基础设施层负责。

落实结果

业务领域层依赖文件抽象，不依赖具体存储实现，后续切换存储方案时不需要改变核心业务模型，符合依赖倒转原则。

四、检查结论

统计项	数量
AI 生成设计中发现的问题总数	5
S - 单一职责问题	1
O - 开闭原则问题	1
L - 里氏替换问题	1
I - 接口隔离问题	1
D - 依赖倒转问题	1

本次检查中最关键的问题是订单状态流转不完整。约拍服务并不是普通商品交易，必须表达取消、待交付、仲裁、返修和退款等分支。核心类图通过 10 个订单状态、订单状态日志、申诉实体、交付轮次和资金状态共同保证交易闭环完整。

核心类图保留模块化单体和分层架构，领域对象负责自身状态和核心规则，跨模块流程由 Service 层组织，文件存储、支付、通知等外部能力通过接口隔离在基础设施层。该设计能够支撑 P0 优先级功能，并与 API 规范、ER 图和建表 SQL 保持一致。